



[BIT703 Web Technologies](#) / [Contents](#) / Topic 10: Server-side web application development with procedural and object-oriented database programming / Database operations in PHP

Create using the OOP approach 4m

The **create** operation performs the **insert** function, used to create new records. The following example shows how to insert new records into the database using the OOP approach. An HTML form is used to submit data using the POST method.

Some of the PHP commands that are used for the OOP approach to insert a new record are listed below:

- **DbConfig** – A simple database abstraction class (found in `classes\DbConfig.php` mentioned below) and database connection created within the `__construct()` method. A new object is created for the class.
- **mysql_escape_string** – Escapes special characters in a string for use in an SQL query, taking into account the current character set of the connection. This function is used to create a legal SQL string that can be used in an SQL statement. In OO PHP, **mysqli::escape_string** is an alias for **mysql_escape_string**.

For inserting data, a simple execute (refer to `classes\Crud.php` file in the archive) function is created. This is used to execute the SQL queries. It does not include the previous prepared SQL statement code from the earlier example (`mysqli_prepare`, `mysqli_stmt_bind_param`, `mysqli_stmt_execute` and `mysqli_stmt_close`) to keep the example simple to read. You can call this function as many times as you like. One of the advantages of OOP is the reusability of the code.

[Skip to main content](#)

Classes as additional files

The `classes\Crud.php` file (in the `CRUD_Topic_10.zip` folder) contains the `Crud` class, which inherits the database connection data, meaning you only need to import this one file to use it. Open the above file in your editor so that you can examine the class closely to see how it was created and how it is used below. We will be using this file for the remainder of the OOP examples. Be sure to verify that the credentials in the `DbConfig.php` file match the ones used for your local database server.

In the same folder, you will also find `classes\Validation.php`. As the file name suggests, this contains a utility class that provides several validation methods. Examine this code too to see how the class has been written and how you can add to it to suit your own application. For example, you can extend the class to include validators and checks for the PHP superglobals `$_GET` and `$_POST`. For brevity purposes, this utility class has not been used in the examples below. Emphasis has been placed on the PHP database operations.



Copy

[Skip to main content](#)

```
1  <?php
2  //including the database connection file
3  include_once("classes/Crud.php");
4  class FormData {
5      public $name;
6      public $address;
7      public $category;
8      __construct($_name,$address,$_category) {
9          $self->name = $_name;
10         $self->address = $_address;
11         $self->category = $_category;
12     }
13 }
14
15 $R = new FormData("SWEET PANCAKES","345 BROADWAY ROAD","DESSERT");
16 $crud = new DbConfig(); //instantiate the DB object
17
18 $result = $crud->execute("INSERT INTO tbl_restaurant(resta_name, resta_address, resta_ca
19 echo "<font color='green'>Data added successfully.";
20 echo "<br/><a href='index.php'>View Result</a>";
21 ?>
```

gistfile1.txt hosted with ❤ by GitHub

[view raw](#)

Thoughts on this page?

